

Detecção de Deadlocks e Violação da Ordem de Aquisição de Travas

Baseado em DEFINITION.md

Universidade Federal do Rio Grande do Norte

16 de julho de 2025

Integrantes

Nome	E-mail
Corpo docente	
Samuel Xavier de Souza	samuel.xavier@ufrn.br
Wedson Torres de Almeida Filho	wedson.almeida@dca.ufrn.br
Corpo discente	
Vinicius de Lima Duarte Sampaio	vinicius.lima.115@ufrn.edu.br
Emanuel Rawa Gurgel de Araujo Freitas	emanuelrawa@gmail.com
Arthur Jose Arruda Skeete	arthurjoskeete@gmail.com
Yuri Maximiliano Brasileiro Santos	yurimax322@gmail.com
José Henrique Targino Dias Góis	ze.gois.00@gmail.com

Sumário

- 1 Introdução e Objetivo
- 2 Fundamentos Teóricos
- 3 Implementação por Arquivo
- 4 Exemplo de Execução
- 5 Conclusão e Extensões

Objetivo Geral

- Estender mutexes didáticos para detectar deadlocks e violações de ordem.
- Inspirado no lockdep do Linux.
- Abordagens: Grafo de espera (reativo) ou verificação de ordem (preventivo).

Requisitos Principais

- Base em mutex C.
- Estruturas: Dono atual, fila de espera, ordem por thread.
- Implementar pelo menos uma abordagem (extra para ambas).
- Foco aqui: Lockdep-style (preventivo); grafo de espera como extensão.

Conceitos Chave

- Deadlock: Espera circular (condições de Coffman).
- Grafo de Espera: Nodes=threads/resources; ciclo=deadlock real (DFS).
- Lockdep-Style: Grafo dependências (nodes=locks, edges=ordem); ciclo=potencial inversão.

lockdep.h: Estruturas de Dados

- `lock_node_t`: Grafo (children) + lista (next); dono (`current_held_lock`).
- `adjacency_locks_t`: Edges para dependências.
- `held_lock_t`: Pilha de holds com trace e timestamp.
- `thread_context_t`: Contexto thread com pilha held.

lockdep.h: Estruturas de Dados

- lock_node_t: Grafo (children) + lista (next); dono (current_held_lock).

```

1 // Node representing a lock in the lock dependency graph.
2 typedef struct lock_node {
3     const void* lock_addr;           // Address of the lock.
4     char lock_name[LOCKDEP_MAX_LOCK_NAME]; // Symbolic name for the
lock.
5     bool was_visited;                // Flag to indicate if
the lock was visited during DFS.
6     adjacency_locks_t* children;      // List of adjacent (ch
locks.
7     struct lock_node* next;          // Next lock node in th
list.
8     held_lock_t* current_held_lock;   // Reference to current
held_lock_t if owned.
9 } lock_node_t;

```

lockdep.h: Estruturas de Dados

- adjacency_locks_t: Edges para dependências.

```
1 typedef struct adjacent_locks {  
2     lock_node_t* lock;           // Pointer to the adjacent lock node  
3     struct adjacent_locks* next; // Next adjacency in the list.  
4 } adjacency_locks_t;
```

lockdep.h: Estruturas de Dados

- held_lock_t: Pilha de holds com trace e timestamp.

```

1 // Represents a lock currently held by a thread.
2 typedef struct held_lock {
3     lock_node_t* lock;                                // Pointer
to the held lock node.
4     struct timespec acquired_at;                      // When th
lock was acquired by this thread.
5     void* acquisition_stack[LOCKDEP_MAX_STACK_DEPTH]; // Stack
trace where acquired.
6     int stack_depth;                                  // Number
of valid stack frames.
7     thread_context_t* thread_context;                // Referen
to the thread context holding this lock.
8     struct held_lock* next;                          // Next he
lock in the list.
9 } held_lock_t;

```

lockdep.h: Estruturas de Dados

- `thread_context_t`: Contexto thread com pilha held.

```
1 // Context information for a thread, including held locks.
2 typedef struct thread_context {
3     pthread_t pthread_id;           // pthread identifier (for internal
4                                     tracking).
5     pid_t thread_id;                // Kernel thread ID (for user-frien
6                                     display).
7     held_lock_t* held_locks;        // List of locks currently held by
8                                     the thread.
9     char thread_name[32];           // thread name for debugging.
10    struct thread_context* next;     // Next thread context in the list.
11 } thread_context_t;
```

pthread__interpose.c: Hooking Pthread

- Interposição via dlsym para lock/unlock/trylock.
- Chama lockdep__acquire/release; evita recursão (in__interpose).
- Para trylock: Preve se deadlock detectado.

```
1   init_real_functions();
2 }
3
4 /// The lockdep uses a mutex to protect its internal state, so we use this
5 to
6 /// avoid recursing lockdep validation across itself.
7
8 static __thread bool in_interpose = false;
9
10 int pthread_mutex_lock(pthread_mutex_t* mutex)
11 {
12     init_real_functions();
```

lockdep_core.c: Lógica Central

- `acquire_lock`: Registra, adiciona dep, checa ciclo (DFS).
- `has_cycle`: DFS recursivo para detectar ciclos; reporta com traces.
- `print_cycle`: Relatório detalhado (path, owners, sugestão order).

```

1      "=====
2      "[LOCKDEP] *** DEADLOCK DETECTED ***\n"
3      "=====
4
5      fprintf(stderr, "\n[LOCKDEP] Deadlock cycle detected:\n");
6      for (size_t i = 0; i < cycle_path_size; i++) {
7          lock_node_t* current_lock = cycle_path[i];
8          fprintf(stderr, "    [%zu] %s\n", i + 1, current_lock->l
9
10         if (current_lock->current_held_lock) {
11             held_lock_t* held = current_lock->current_held_lock

```

t05_dining_philosophers.c: Exemplo

- Clássico problema: 5 filósofos, 5 garfos (mutexes).
- SHALL_LOCK=1: Todos pegam left first → ciclo.
- Com lockdep: Detecta preventivamente.

```
1      if (id == NUM_PHILOSOPHERS - 1) {
2          // Last philosopher picks up right fork first
3          first_fork = right_fork;
4          second_fork = left_fork;
5      } else {
6          // Other philosophers pick up left fork first
7          first_fork = left_fork;
8          second_fork = right_fork;
9      }
10 }
```

Resultado no Exemplo

- Com LD_PRELOAD: Detecta ciclo (e.g., lock_0x8050 → ... → lock_0x8050).
- Reporta: Path, tempos, traces, states, sugestão (order by addr).
- Atende preventivo; extensão para reativo via waiters.

Conclusão

- Atende lockdep-style; demonstra grafos em concurrency.
- Limitações: $O(n)$ lookups (add hash); sem fila wait full.
- Impacto: Educacional; usa IA para análise.

Extensões Futuras

- Implementar grafo espera: Add fila waiters, DFS thread-lock.
- Suporte a outros sync (condvars, RW locks).
- Otimizações: Hash para lookups; integração gdb/valgrind.

Perguntas?

Obrigado! Dúvidas?